

Yleinen lisäarvontuottamisen malli

Organisaatioiden toiminnan analysointimenetelmä

Tomi Lind

Tiivistelmä

Kirjoittajat:	Tomi Lind	
Tutkielman nimi:	Yleinen lisäarvontuottamisen malli Organisaatioiden toiminnan analysointimenetelmä	
Päiväys:	7.7.2020	Sivuja: 34
Tämä tutkielma on tehty Tomi Lindin henkilökohtaisena kehitystyönä tietotekniikan palveluliiketoiminnan yhteydessä vuodesta 1999 alkaen tehtyjen havaintojen pohjalta. Ensimmäinen kirjalliseen julkaisun mallista Tomi Lind esitteli 15.8.2001 Fujitsu-Siemens Professional Services Finland liiketoimintajohdolle, nimikkeellä ”Hosting Business” (Lind, 2001). Mallin tavoitteena oli esitellä IT-palveluliiketoiminnan organisaatio ja sen toiminnot riippuvuuksineen. Tutkielman käsittelee mallia tietotekniikan palvelualan liiketoimintojen rakenneteen näkökulmasta sekä esittelee menetelmät sekä organisaation rakenteen kuvaamiseksi ja dokumentoimiseksi että organisaation tuottaman lisäarvon analysoimiseen. Tutkielmasta rajattiin pois mallin tarjoamien kuvausten ja dokumentointimenetelmien kuvaukset. Tutkielman lopputuloksena esitellään toimialasta riippumaton organisaatiorakenteen ja -toiminnan kaikenkattava analysointimalli. Johtopäätöksenä suositellaan, että esitellyn analysointimallin soveltuvuutta ja saavutettavia hyötyjä tutkitaan syvällisemmin. Tutkimusmenetelmänä suositellaan erilaisten organisaatioiden rakenteiden ja toimintojen analysointia mallin mukaisesti.		
Avainsanat:	Yritys, liiketoiminta, yhteisö, yhdistys, hallinto, koulutus, kompetenssi, dokumentaatio, asiakaspalvelu, myynti, tuotanto, tuotteistus, kehitys, tutkimus, budjetointi, talous, kannattavuus, analysointi	

Sisällysluettelo

Tiivistelmä	2
Sisällysluettelo	3
1 Johdanto	5
1.1 Tutkielman tausta	5
1.2 Tutkimusongelma	5
1.3 Tutkielman tavoitteet	5
1.4 Tutkielman rajaukset	5
1.5 Tutkimusmenetelmät	6
2 Aikaisemman tiedon kuvaus	7
3 Tutkimustulokset	8
3.1 Sovelluksen yleiskuvaus	8
3.2 Reittien syöttäminen palveluun	8
3.2.1 Www-lomake	8
3.2.2 WAP-lomake	9
3.3 Palvelun käyttö	10
3.3.1 Automaattinen hälytys (push-palvelu)	10
3.3.2 Tilanne reitillä	11
3.3.3 Pisteiden tilanne	12
3.3.4 Reitien historiatiedot	13
3.4 WAP-palvelin	14
3.4.1 Tekninen toteutus	14
3.4.2 Staattiset sivut	14
3.4.3 Dynaamiset sivut	14
3.5 Tieinfo-sovellus	15
3.5.1 Käyttöympäristö ja toteutustyökalut	15
3.5.2 Mittaustuloksia kuvaava tietorakenne	15
3.5.3 Käyttäjien kuvaava tietorakenne	17
3.5.4 Www-datan parserointi tietokantaan	19
3.5.5 Historiatiedon päivittäminen	20
3.5.6 Historiatiedon luku	21
3.5.7 Hälytysten antaminen	21
3.5.8 Käyttäjän esittämiin kyselyihin vastaaminen	22
3.5.9 Skaalautuvuustarkasteluja	23
3.6 Palvelun toimivuus ja havaitut seikat	23
4 Johtopäätökset	25
5 Suositukset	26
5.1 Personointi	26
5.2 Tekstiviestien ja puhepostilaatikon käyttö	26
5.3 Hinnoittelumalleja	26
5.4 Tukiasematiedon käyttäminen	27

6 Tutkielman arviointia	28
Lähdeluettelo.....	Error! Bookmark not defined.
Liitteet	30
Liite A, Kuva Tielaitoksen www-palvelusta	30
Liite B, Otteita Tieinfon rekisteritiedoista	31
Liite C, Otteita Tieinfon ohjelmakoodista	32

1 Johdanto

1.1 Tutkielman tausta

Pääkaupunkiseudulla autoilevalle ei tällä hetkellä tarjota luotettavaa ruuhkatietoa. Radion RDS-palvelun avulla saadaan välitettyä liikennetiedotteita, mutta personoinnin puuttuessa niissä keskitytään vain suurimpiin valtavyliä tukkiviin onnettomuuksiin, ei jokapäiväisiin ruuhkautumisiin. Muutaman vuoden takaiset säännölliset helikopteriraportit klaudekoptereineen, maramoottoreineen ja heikkiseppineen ovat kadonneet.

Tarve yksilölliselle mobiilille ruuhkapalvelulle on olemassa, sillä ruuhkatilanne vaihtelee suuresti päivästä päivään ilman sellaista syytä, jonka autoilija voisi ennakoida. Asiakaspotentiaalista saa jonkinlaisen käsityksen esimerkiksi Kehä I:llä aamuruuhkan aikaan. Monissa tapauksissa tukkeutunut tieosuus olisi kierrettävissä, kunhan riittävästi oikea-aikaista tietoa olisi saatavilla.

Tielaitos mittaa liikenteen sujumista pääkaupunkiseudun valtavyillä ja julkaisee mittaustulokset [www-sivuillaan](#) lähes reaaliaikaisena. Tämä tieto on käytännössä autoilijan tavoittamattomissa mobiilien [www-päätelaitteiden](#) harvinaisuuden ja autokäyttöön soveltumattomuuden vuoksi.

Yhdistämällä liikennetilanteen WAP-puhelimen sijaintiin ja suunniteltuun ajoreittiin, voidaan asiakkaalle tarjota yksilöllinen ruuhkatiedote.

1.2 Tutkimusongelma

Miten uusien matkapuhelintekniikoiden avulla voidaan toteuttaa sellainen palvelu, joka tarjoaa autoilijalle tietoa matkareitillä olevista hidasteista.

1.3 Tutkielman tavoitteet

Tavoitteena on toteuttaa prototyyppijärjestelmä, jonka avulla voidaan esitellä ruuhkatiedotteiden välittämistä asiakaskohtaisesti personoituna palveluna. Vaikka prototyyppi onkin suljettu yhden käyttäjä järjestelmä, sen perusratkaisut pyritään tekemään sellaisiksi, että palvelua voidaan tarvittaessa skaalata suuremmille käyttäjämäärille.

1.4 Tutkielman rajaukset

Tutkimuksessa luodaan demo-versio palvelusta yhdelle käyttäjälle (yhdelle puhelinnumerolle). Automaattinen varoitus toteutetaan käyttäjän kyselyyn perustuen, sillä nykyinen WAP 1.1 standardi ei vielä tue automaattisia ”push”-tyyppisiä palveluita. Demossa tarvittava personointitieto tallennetaan tekstitiedostoihin, jotka luetaan

sovellusta käynnistettäessä. Todellisessa tuotantokäytössä käyttäjä voi personoida palvelua reaaliaikaisesti. Demossa asiakkaan puhelinnumero on kiinteä, mutta tuotantoversiossa puhelinnumero saadaan WAP-yhdyskäytävältä.

Tuotantokäytössä asiakas syöttäisi omat reittinsä palveluun joko www-lomakkeella tai WAP:n kautta. Demossa näitä toimintoja ei toteuteta. Sen sijaan pohditaan erilaisia toteutusvaihtoehtoja niiden käyttöliittymiksi.

Demo suunnitellaan sellaiseksi, että sitä voitaisiin käyttää materiaalina käytettävyyystutkimuksessa, mutta varsinaista käytettävyydestä ei tehdä. Liiketoimintamallin kehittäminen edellyttäisi käytettävyydestä tutkimuksia, joten niiltäkin osin tutkielmassa tyydytään esittämään erilaisia mahdollisuuksia kaupallisen palvelun toteuttamiseksi.

1.5 Tutkimusmenetelmät

Sekä tutkielman osana luotava demo ja itse tutkielma jakautuu loogisesti kahteen osaan. Toinen osa käsittelee palvelua loppukäyttäjän näkökulmasta ja toinen sen takana olevaa teknistä toteutusta.

Tekniseen toteutukseen liittyy parseri- ja tietokantasovelluksen ”TieInfo” luonti, joka hakee tiedot tiehallituksen www-palvelimelta ja tallentaa ne tietokantaansa. Pääosa prosessoinnista, esimerkiksi hälytysten asettaminen ja määrämuotoisiksi tekstitiedostoiksi tulostaminen, tehdään TieInfo-sovelluksessa. TieInfossa on otettu huomioon skaalautuminen: tehdyillä ratkaisuilla näytetään esimerkinomaisesti, millaisella rakenteella voidaan tehokkaasti palvella suurtakin yhtäaikaista käyttäjämäärää.

Tutkielman loppukäyttäjän näkökulmaan liittyy palvelun demon käyttöliittymän luonti, staattiset wml-sivut ja perl-skriptit, jotka luovat dynaamisesti liikennetilanteen esittämät wml-sivut. Demossa puhelinnumero on kiinnitetty skriptien kutsuihin, tuotantoversiossa puhelinnumero saadaan WAP-yhdyskäytävältä.

2 Aikaisemman tiedon kuvaus

Erilaisia GPS:ään perustuvia autonavigointijärjestelmiä on maailmalla lukuisia /Blaupunkt, Philips/. Lukuisissa eurooppalaisissa hankkeissa on kehitetty monikielisen ajantasaisen liikennetiedon välittämistä radioasemien RDS-TMC (ja DAB)-lähetysten mukana /RDS/. RDS-TMC-europrojekteja ovat olleet ARTS, CENTRICO, CORVETTE, SERTI ja VIKING.

Kansallisista RDC-TMC –projekteista YLE:n ja Tielaitoksen projektin oli määrä kattaa koko Suomi vuoden 1999 loppuun mennessä. Vastaanottimia ei tosin ole laajalti kaupan.

EU:n Cordis –hankkeen Promise-projektissa on tutkittu WAP-tekniikan hyödyntämistä mobiilin käyttäjän tietopalveluna /Kopitz et al/. Promisen pääpaino on ollut kuitenkin muualla kuin autoilijan ruuhkatiedotteissa. /Stakes/

3 Tutkimustulokset

3.1 Sovelluksen yleiskuvaus

Prototyypijärjestelmässä asiakas määrittelee sovellukseen työmatkareittinsä, joka jää palvelun muistiin. Asiakkaan tulee myös määritellä, mihin aikaan hän arvioi ohittavansa reitin varrella olevat mittapisteet. Palvelu varoittaa, jos edessä on tavallista vaikeampi liikennetukos. Lopullisessa palvelussa sijainti saataisiin joko GSM-verkon tukiasematietona tai arvioituun lähtöaikaan ja ajonopeuteen perustuen.

Liikennetilanteesta määritellyllä reitillä laaditaan WAP-käyttöliittymäkortti, joka lähetetään asiakkaan puhelimeen, mikäli asiakas on ajamassa liikennetukokseen. Automaattinen varoitus lähetetään asiakkaalle vain siinä tapauksessa, että tukos on olemassa. Lisäksi palvelun kautta voidaan hakea historiaan perustuva liikennearvio tai vallitseva tilanne mistä tahansa mittapisteestä personoidun reitin varrelta.

Jotta skaalautuvuustavoite saataisiin täytettyä, sovellukseen rakennettiin oma tietokanta, johon tallennetaan määrävälein Tielaituksen keräämä mittausdata. Tällöin suurikaan käyttäjämäärä ei rasita tiedon tuottajan järjestelmiä, vaan kuormitus kohdistuu itse sovellukseen. Tietokantasovellus puolestaan toteutettiin siten, että monen käyttäjän tarvitsema tiedonkäsittely tehdään laskennallisesti tehokkaalla tavalla.

Samaan tietokantaan rakennettiin myös käyttäjäkohtainen personointi, jonka avulla koko tietomäärästä suodatetaan asiakkaan tarvitsema informaatio aika- ja paikkasidonnaisesti. Lisäksi tietokanta pitää yllä mittapiste- ja kellonaikakohtaista historiatietoa ja niistä johdettuja tunnuslukuja.

Data haetaan tietokantaan Tielaituksen www-sivua aika ajoin parseroimalla. Luettelo saatavilla olevista mittapisteistä samoin kuin asiakaskohtainen personointitieto tuodaan demopalveluun tekstitiedostoina, jotka luetaan palvelun käynnistämisen yhteydessä.

Seuraavassa luvussa kuvataan demojärjestelmän lisäksi niitä toimintoja, joita voitaisiin ajatella tuotantojärjestelmässä olevan.

4.23.2 Reittien syöttäminen palveluun

3.2.1 Www-lomake

Kun asiakas tilaa palvelun hänelle voidaan antaa salasana, jonka avulla hän voi lähettää tiedot hänen reiteistään palveluun. Salasana kysytään, kun asiakas lataa reittien muokkausta varten luodut sivustot. Toisena vaihtoehtona on kysyä salasana vasta www-lomakkeen tallennuksen yhteydessä. Palvelun reitit ovat aina sidottu tiettyyn puhelinnumeroon, joka toimii samalla käyttäjätunnuksena.

Jokaiselle käyttäjälle tulee oma pääsivu, jolta näkyy kaikki asiakkaan reitit. Tältä sivulta asiakas voi valita reitin muokkauksen, lisäyksen tai poiston. Kun reittejä halutaan luoda

Formatted: Bullets and Numbering

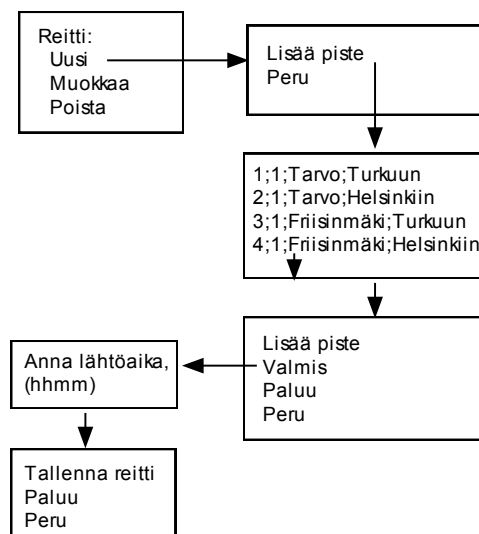
tai muokata, niin avataan kyseiselle reitille oma sivu. Tällä sivulla asiakas voi nimetä reitin, ponnahdusvalikoista valita oman reittinsä pisteet ja antaa arvioidun lähtöajan. Toisena vaihtoehtona on antaa reitti osoittamalla reitin pisteet suoraan kartalta. Tämä on käyttäjälle nopeampi vaihtoehto, mutta sen luominen teknisesti hyvin toimivaksi voi olla vaikeaa.

Palvelu on suunniteltu personoitavaksi myös sen esittämän sisällön ja valitun hälytysrajan suhteen. Asiakas voi valintaneliöitä rastittamalla valita hänen näytölleen tietystä palvelusta annettavat tiedot. Hälytysraja asetetaan nopeuden hidastumisena normaalitilanteeseen verrattuna. Hälytysrajan on syytä olla riittävän suuri, esimerkiksi liikenteen nopeuden hidastuminen mittapisteessä 30% normaalista, sillä liian pieni arvo voi johtaa jokapäiväiseen hälytykseen reitillä. Hälytystä ei kuitenkaan suoriteta, vaikka liikenne olisikin hidastunut annettua raja-arvoa enemmän, jos se on edelleen sujuvaa.

4.1.23.2.2 WAP-lomake

Reittitiedon päivittäminen WAP:n kautta on pienen näytön vuoksi väistämättä hitaampaa ja hankalampaa kuin webin kautta. WAP:n etuna on kuitenkin tunnistuksen perustuminen puhelinnumeroon, käyttäjätunnusta ja salasanaa ei tarvita, sillä reittejä voi päivittää vain siitä numerosta, jolle palvelu on tilattu.

Personointitiedon päivittäminen WAP:n kautta johtaa väkisinkin melko syvään valikkoon.

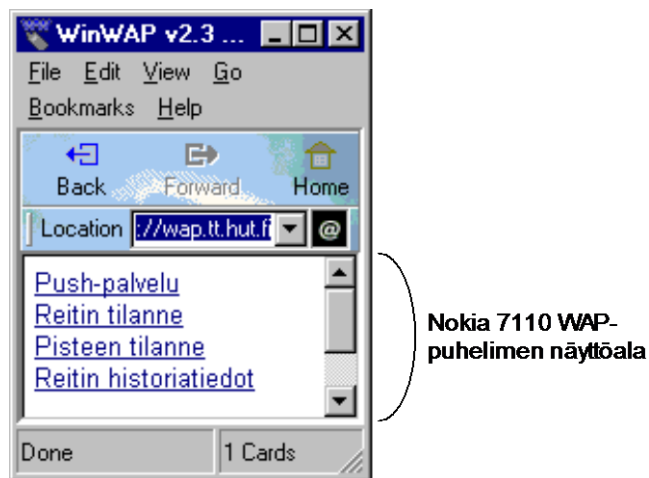


Kuva 1. Esimerkki mahdollisesta reitin tallennuksen valikkorakenteesta.

Formatted: Bullets and Numbering

3.3 Palvelun käyttö

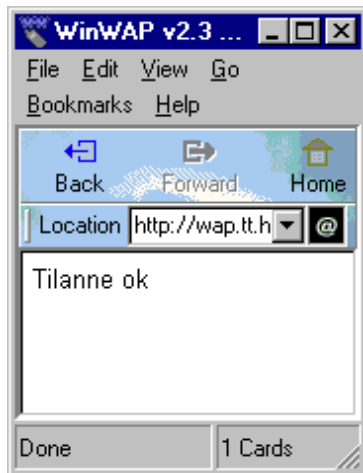
Haluttu palvelu valitaan pääsivun neljästä eri vaihtoehdosta. Automaattinen hälytys on yhtenä valintana, sillä sen toteuttaminen ei ole vielä nykyisellä tekniikalla (WAP 1.1) mahdollista. Käyttöliittymän perusajatuksena on ollut yksinkertaisuus ja matala valikkorakenne. Käytännössä kuitenkin pitkän valintalistan vierittäminen pienellä näytöllä (Nokia 7110, neljä riviä) on hidasta ja hankalaa. Siksi joihinkin toimintoihin on lisätty taso listan lyhentämiseksi.



Kuva 2. Palvelun etusivu WinWAP-selaimella katsottuna. Ikkunan koko vastaa Nokia 7110 WAP-puhelimen näytön kokoa.

3.3.1 Automaattinen hälytys (push-palvelu)

Kun demossa käyttäjä valitsee automaattisen hälytyksen (push-palvelu), niin palautetaan tieto "Tilanne ok", jos hälytysrajaa ei ole ylitetty. Mikäli hälytys on päällä, niin palautetaan "Ruuhkaa edessä" tiedon lisäksi hälyttävän pisteen tai pisteiden tiedot.

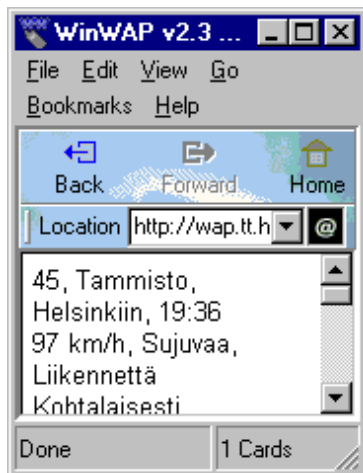


Kuva 3. Automaattisen hälytyksen testaus demolla.

Tulevaisuuden mahdollisessa tuotantoversiossa tämä toiminto on automaattinen. Kun asiakas on tehnyt tarvittavat valinnat personointitietokantaan, niin hänelle lähetetään hälytys, kun valittu ruuhkataso aktiivisella reitillä on tosiasia. Kun hälytys on annettu, niin se merkitään tietokantaan, jottei samasta asiasta tule useita hälytyksiä.

3.3.2 Tilanne reitillä

”Tilanne reitillä”-valinta palauttaa tilannetiedot pyyntöhetkellä aktiivisen reitin pisteillä. Aktiivinen reitti saadaan kellonajan perusteella.

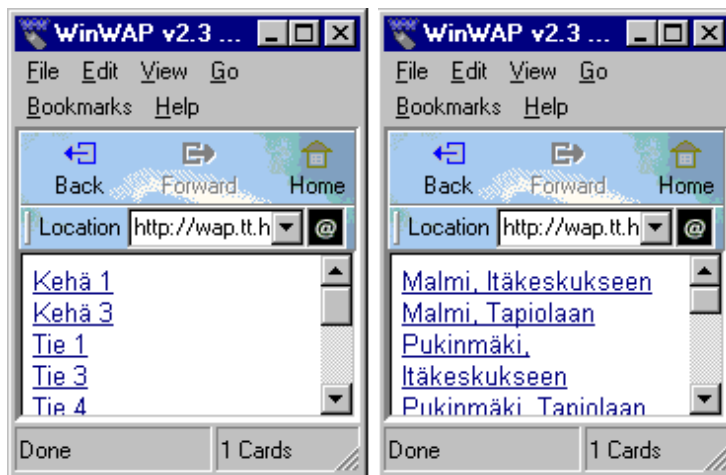


Kuva 4. Tilanne reitillä kyselyhetkellä.

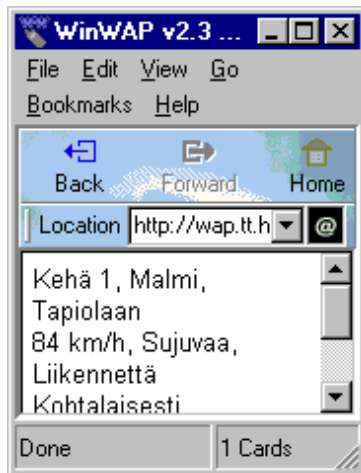
Tässä vaihtoehtoisesti voisi myös olla mahdollisuus valita reitti, jonka tiedot halutaan. Tämä voi olla tarpeen silloin kun asiakkaalla on tavanomaisesta poikkeava päiväohjelma, esimerkiksi työvuoro on vaihtunut. Tämän vaihtoehtoisen toiminnon voi tarvittaessa lisätä päävalikkoon otsikolla ”Valitun reitin tilanne”.

3.3.3 Pisteen tilanne

Pääkaupunkiseudulla on jo 42 eri mittapistettä, joten tässä toiminnossa ensin valitaan tie ja sitten mittapiste kyseiseltä tieltä. Tässä toiminnossa yksi lisätaso valikkoon on valittu siksi, että kaikkien pisteiden esittäminen yhtenä listana johtaa puhelimen pienellä näytöllä liian pitkään listaan.



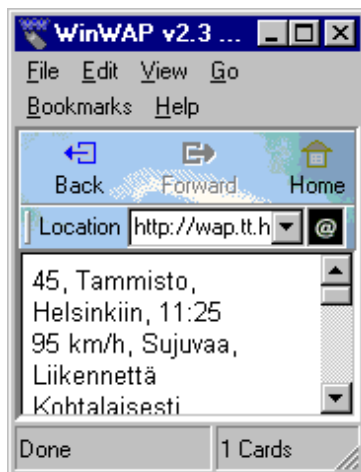
Kuva 5. ”Pisteen tilanne” –toiminnon valinnat. Kuvassa ensin on valittu Kehä 1 ja sitten ”Malmi, Tapiolaan”. Tuloste näkyy kuvassa 6.



Kuva 6. Valitun pisteen liikennetilanne kyselyhetkellä

3.3.4 Reitin historiatiedot

Tällä toiminnolla asiakas voi tarkastella reitillä vallinnutta tilannetta valittuna ajankohtana.



Kuva 7. Kuvassa on reitillä kello 11:25 vallinnut tilanne, kun kyselyn aikana kello oli 19:50.

3.4 WAP-palvelin

3.4.1 Tekninen toteutus

WAP-palvelin on tavallinen www-palvelin, johon on määritelty WAP-palvelun vaatimat MIME-tyypit wml-sivustoille.

```
WML          text/vnd.wap.wml wml
WMLScript    text/vnd.wap.wmlscript wmls
.wml         text/vnd.wap.wml
.wmls        text/vnd.wap.wmlscript
.wbmp        image/vnd.wap.wbmp
```

Palvelun asiakas soittaa johonkin WAP-yhdyskäytävään (demossa Radiolinja, puh 10050), jonka kautta hän voi ottaa yhteyden itse palveluun <<http://wap.tt.hut.fi/>>. Kun datan haku- ja parserointiohjelma päätettiin toteuttaa Windows NT 4.0 ympäristössä, niin oli luonnollista valita palvelinkoneeseen sama ympäristö. Palvelin sovellukseksi valittiin Savant-ohjelmisto, missä on riittävä tuki CGI-skripteille ja MIME-tyyppien määrittämiselle. Kaikkia palvelun ohjelmistoja ajetaan yhdessä internetiin kytketyssä PC-koneessa, jossa on 350MHz Pentium II prosessori, 10GB kovalevy ja 128MB keskusmuisti sekä 100Mbps ethernet kortti verkkoyhteyttä varten.

4.1.23.4.2 Staattiset sivut

Palvelun etusivu ”index.wml” on staattinen wml-sivu. Muutoin palvelu perustuu pääosin CGI-tekniikalla dynaamisesti luotaviin wml-sivuihin. CGI-skriptit toteutettiin perl-kielillä. Perl-kielen etuna on sen siirrettävyys ympäristöstä toiseen. Tämä oli tarpeen, sillä pääosa perl- ja wml-kehityksestä tehtiin Macintoshilla.

Staattiset sivut ovat varsin yksinkertaisia wml-sivuja. Koska WML perustuu XML:ään, niin jokaisen sivun ensimmäiset rivit kertovat XML-version ja sen mistä löytyvät määrittäykset käytetylle kielelle.

```
<?xml version="1.0"?>
<!DOCTYPE wml PUBLIC "-//WAPFORUM//DTD WML 1.1//EN"
"http://www.wapforum.org/DTD/wml_1.1.xml">
```

4.1.33.4.3 Dynaamiset sivut

Koska kyseessä on personoitava ja sisällöltään jatkuvasti muuttuva palvelu on kaikki varsinaista tietoa sisältävät sivut luotava dynaamisesti skriptien avulla. Jokaiselle käyttäjälle saadaan omat henkilökohtaiset sivut tunnistamalla asiakas puhelinnumeron perusteella. Skriptien tulosteen ensimmäisellä rivillä palautetaan asiakkaan selaimelle tieto siitä, että sivut ovat wml-sivuja.

```
print "Content-type: text/vnd.wap.wml\n\n";
```

Muutoin palautettava data vastaa staattisia wml-sivuja. Demossa puhelinnumero on kiinnitetty skriptien kutsuihin, mutta tuotantoversiossa se saadaan WAP-yhdyskäytävältä.

Formatted: Bullets and Numbering

Formatted: Bullets and Numbering

Palvelun kyselyissä skriptit ohjaavat TieInfo-sovellusta komentotiedostoilla. Komentotiedostojen käyttö on selvitetty TieInfo-ohjelmistoa käsittelevän luvun 3.5 luvussa 3.5.7. TieInfo palauttaa komenttoon vastauksen, jonka perusteella luodaan palautettava wml-sivu. Vastauksissa on yhdellä rivillä yhden mittapisteen tiedot. Perlin split()-funktioilla saadaan rivillä olevat arvot sijoitettua muuttujiin.

3.5 Tieinfo-sovellus

3.5.1 Käyttöympäristö ja toteutustyökalut

Tieinfo toteutettiin 32-bittisenä Windows-sovelluksena, jonka ohjelmoinnissa käytettiin Microsoft Visual C++:n versiota 6.0. Tieinfossa tietokantapalvelin ja www-parseri ohjelmoitiin yhdeksi monisäikeiseksi sovellukseksi, jonka reaaliaikainen tiedonkäsittely – palvelupyyntöihin vastaaminen - tapahtuu kokonaisuudessaan keskusmuistissa. Demojärjestelmässä Tieinfoa ajetaan Windows NT 4.0 –käyttöjärjestelmässä.

Tieinfossa hyödynnettiin C++:n tarjoamia olio-ohjelmointimahdollisuuksia. Tieinfossa ei käytetä taulukkomuotoista tiedon esitystä, vaan taulukot on korvattu linkitetyillä listoilla, historiatietoja lukuun ottamatta. Linkitettyjä listoja käytetään skaalautuvuuden helpottamiseksi. Rakenteiden yksinkertaisuuden vuoksi Tieinfo.exe on hyvin kompakti, kooltaan vain 48 kilotavua. Lähdekoodi on aihepiireittäin jaoteltuna 17 tiedostossa, joista näyte löytyy liitteestä C.

Muutamaa perusteltavissa olevaa poikkeusta lukuun ottamatta kaikki konfigurointiparametrit on sijoitettu Windowsin rekisteriin (liite B), josta niitä voidaan tarvittaessa muuttaa.

4.4-23.5.2 Mittaustuloksia kuvaava tietorakenne

Mittapisteyden ryhmittelyllä noudatetaan samaa kolmitasoista hierarkiaa kuin Tielaitoksen www-palvelussakin: Useimmilla pääkaupunkiseudun valtaväylillä (1) on useita mittapaikkoja (2). Kullakin mittapaikalla on tyypillisesti kaksi mittapistettä (3), yksi kumpaankin suuntaan.

Tieinfo ei aseta rajoituksia teiden, mittapaikkojen ja mittapisteyden lukumäärään. Niitä kuvaava tietorakenne muodostuu, kun sovellus käynnistyessään lukee mittapisteluettelon, jossa kullekin mittapisteelle on annettu juokseva viitenumero sekä tien ja mittapaikan nimi. Ote mittapisteluettelosta on kuvassa 8. Tällä hetkellä mittapisteyttä on kaikkiaan 42.

Formatted: Bullets and Numbering

23;Kehä 3;Voutila;Itään	Viitenumero
24;Kehä 3;Voutila;Länteen	Tien nimi
25;Kehä 3;Järvenperä;Itään	Mittapaikan nimi
26;Kehä 3;Järvenperä;Länteen	Mittapisteen nimi
27;51;Hanasaari;Espooseen	
28;51;Hanasaari;Helsinkiin	
29;Kehä 1;Malmi;Itäkeskukseen	
30;Kehä 1;Malmi;Tapiolaan	
31;Kehä 1;Pukinmäki;Itäkeskukseen	
32;Kehä 1;Pukinmäki;Tapiolaan	
33;Kehä 1;Länsi-Pakila;Itäkeskukseen	
34;Kehä 1;Länsi-Pakila;Tapiolaan	

Kuva 8. Mittapisteluetteloon on merkitty kunkin mittapisteen oleelliset tiedot

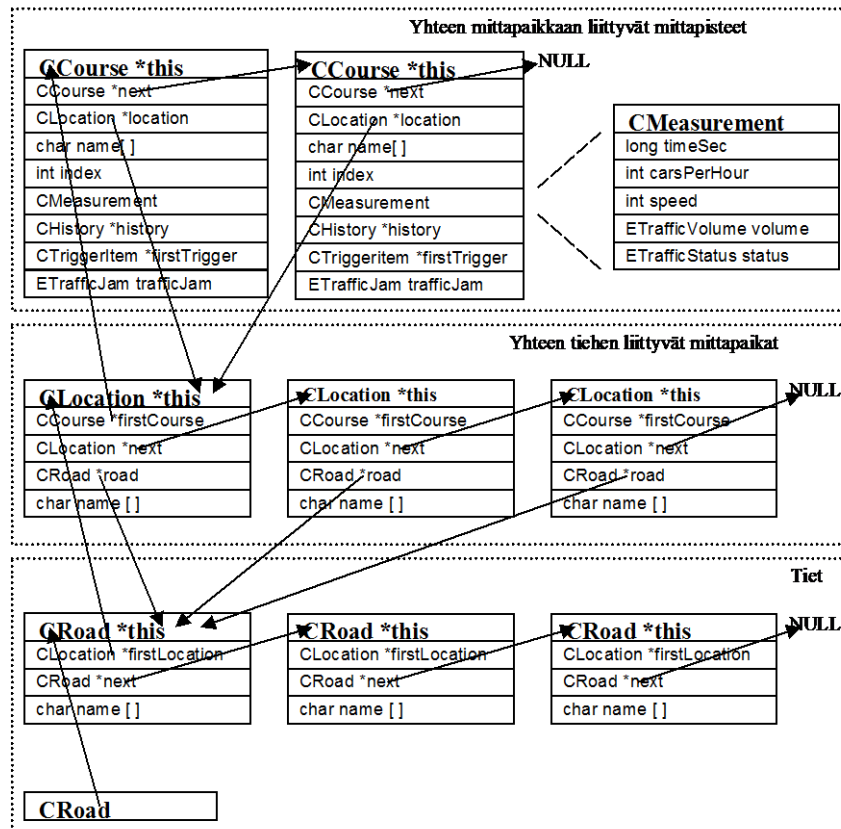
Kun sovellus käynnistyyssä lukee mittapisteluettelon, se muodostaa samaa kolmitasoista hierarkiaa noudattelevan puun, jossa kukin tason haarautuu linkitettyinä listana. Kaikki tiedot keskenään muodostavat siis linkitetyn listan, samoin yhden tien kaikki mittapaikat keskenään ja yhden mittapaikan kaikki mittapisteen keskenään. Rakenne on esitetty kuvassa 9.

Kustakin mittapistestä (kuvassa *CCourse*) on osoitin sen mittapaikan tietorakenteeseen (*CLocation*), johon se kuuluu, samoin kustakin mittapaikasta on osoitin sen tien tietorakenteeseen (*CRoad*), johon mittapaikka kuuluu. Mittapisteluettelosta luetaan myös mittapisteen yksilöivä viitenumero (*index*).

Mittapisteen tietorakenteessa on varattu tilaa mittapistestä kerätyille mittausdatalle (*measurement*) kellonaikoihin, osoitin (*history*) kyseisen mittapisteen historiatietoihin sekä mittapisteen tietojen päivityksen yhteydessä tehty analyysi (*trafficJam*) siitä, onko liikenne jumiutunut liikaa vai ei.

Mittausdata on oma olionsa, joka pitää sisällään Tielaitoksen *www*-palvelusta saatavat tiedot: nopeus (*speed*) ja liikenneviikkaus (*carsPerHour*) sekä sanallinen kuvaus liikennemäärästä (*volume*) ja liikenteen sujuvuudesta (*status*).

Näiden tietojen lisäksi mittapistestä on osoitin (*firstTrigger*) linkitettyyn listaan, jossa on viitteet kaikkiin käyttäjiin, jotka ovat kiinnostuneita kyseisen mittapisteen tapahtumista. Nämä listat yhdistävät kaksisuuntaisesti mittauksia ja käyttäjiä kuvaavat tietorakenteet. Triggereistä on tarkempi selvitys seuraavassa luvussa, käyttäjiä koskevan tietorakenteen yhteydessä.



Kuva 9. Mittapisteiden tietorakenne koostuu teiden (CRoad), mittapaikkojen (CLocation) ja mittapisteiden (CCourse) muodostamasta puusta.

3.5.3 Käyttäjiä kuvaava tietorakenne

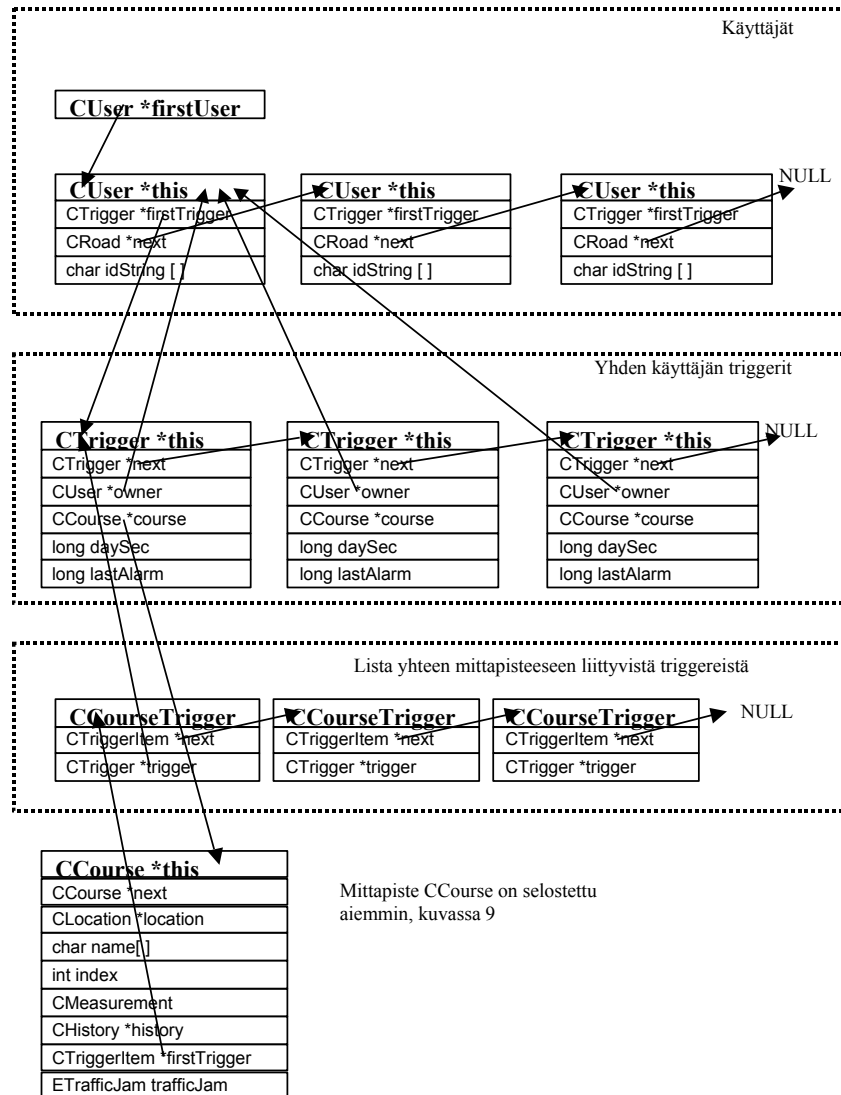
Samoin kuin mittapisteitä, myös käyttäjiä kuvaava tietorakenne luodaan sovelluksen käynnistyessä. Käyttäjiä kuvaavassa tietorakenteessa perusyksikkö on *triggeri*, jossa on määritetty mittapisteen tunnusluku ja arvioitu kellonaika, jolloin käyttäjä ohittaa mittapisteen, sekä käyttäjän tunniste. Tietorakenne luetaan tiedostosta, josta esimerkki on kuvassa 10.

0400606699;20;08:35
0400606699;34;08:40
0400606699;36;08:45
0400606699;38;08:45
koe000001;2;08:20
koe000001;36;08:25
koe000001;4;08:30
koe000001;6;08:35

Käyttäjän tunniste (esim. puhelinnumero)
Mittapisteen viitenumero
Arvioitu ohitushetki, nimellisaika

Kuva 10. Käyttäjäluettelo koostuu käyttäjätunnisteen, mittapisteen ja kellonajan yhdistävistä triggereistä

Kun käyttäjäluettelo luetaan tietokantaan, syntyy sinne kuvan 11 kaltainen tietorakenne. Käyttäjistä muodostuu linkitetty lista, jossa kunkin käyttäjän tietorakenteesta on osoitin käyttäjälle kuuluvien triggerien listan alkuun (*firstTrigger*). Kunkin triggerin tietorakenteesta on osoitin (*owner*) sen käyttäjän tietorakenteeseen, jolle kyseinen triggeri kuuluu, sekä osoitin triggeriin liittyvään mittapisteeseen (*course*). Lisäksi triggerin tiedoissa on nimellinen kellonaika (*daySec*), jolloin käyttäjän oletetaan ohittavan kyseinen mittapiste, ja merkintä (*lastAlarm*) siitä hetkestä, jolloin triggeri antoi viimeisimmän hälytyksensä.



Kuva 11. Käyttäjäluettelon tietorakenne liittyy mittapisteiden tietorakenteeseen triggereiden kautta.

3.5.4 Www-datan parserointi tietokantaan

Mittapisteiden tilanne luetaan Tielaitoksen www-palvelusta (/Tielaitos/, liite A) kerran viidessä minuutissa. Tielaitoksen www-palvelussa liikennetilanne esitetään sekä

havainnollisena karttana että html-muotoisena datana. Parseri lukee liikennetilannesivun html-datan http-protokollaa käyttäen - sellaisilla parametreillä, että kaikki välimuistit ohitetaan (tämä opittiin kantapään kautta). Luettu tiedosto on kooltaan noin 60 kilotavua ja sen parserointi tehdään kolmessa vaiheessa.

Parseroinnissa sivusta etsitään ensin mittapistetietojen alkukohta, jonka yläpuolella oleva html-tieto poistetaan. Toisessa vaiheessa poistetaan kaikki tagit siten, että kunkin mittapaikan tiedot jäävät yhdelle riville. Kolmannessa vaiheessa mittapaikan tiedot jaetaan mittapisteisiin ja luetaan tietokantaan.

Tietojen päivityttyä tietokantaa kirjoitetaan raporttiedosto RAPORT.TXT (hakemisto määriteltävissä rekisteristä), joka sisältää kaikkien mittapisteluettelossa esiintyneiden mittapisteiden senhetkiset tiedot. Ote raporttiedostosta on kuvassa 12.

```
23;0;Kehä 3;Viitenumero;Itään;14:18;Suoritusnopeus;84;1248;Kohtalaisesti
24;0;Kehä 3;Voutila;Länteen;14:18;Suoritusnopeus;84;1248;Kohtalaisesti
25;0;Kehä 3;Järvenperä;Itään;14:23;Suoritusnopeus;98;636;Meikko vähän
26;0;Kehä 3;Järvenperä;Länteen;14:23;Suoritusnopeus;98;636;Meikko vähän
27;0;51;Tie 1;Saari;Espooseen;14:16;Suoritusnopeus;88;1904;Kohtalaisesti
28;0;51;Tie 1;Saari;Espooseen;14:16;Suoritusnopeus;88;1904;Kohtalaisesti
29;0;Kehä 1;Malmi;Itäkeskukseen;14:19;Suoritusnopeus;85;1404;Kohtalaisesti
30;0;Kehä 1;Malmi;Tapiolaan;14:19;Suoritusnopeus;80;1380;Kohtalaisesti
31;0;Kehä 1;Pukinmäki;Itäkeskukseen;14:17;Suoritusnopeus;89;1390;Kohtalaisesti
32;0;Kehä 1;Pukinmäki;Tapiolaan;14:17;Suoritusnopeus;85;1188;Kohtalaisesti
33;0;Kehä 1;Länsi-Pakila;Itäkeskukseen;14:16;Suoritusnopeus;86;2400;Kohtalaisesti
```

Kuva 12. Parseroinnin yhteydessä kirjoitetaan kaikkien mittapisteiden tiedot käsittävä raporttiedosto

Kaikki sovelluksen kirjoittamat tulosteet noudattavat samaa formaattia, jossa kunkin mittapisteen tiedot esitetään yhdellä rivillä puolipistein erotettuna.

3.5.5 Historiatiedon päivittäminen

Kullakin mittapisteellä on oma päivittäinen historiatietonsa, joka on jaettu kymmenen minuutin aikasiivuihin. Aikasiivuja vuorokaudessa on siis 144. Historiatiedot kattavat neljäntoista päivän ajanjakson. Joka kerta, kun käyttöjärjestelmän kello ylittää kahden aikasiivun välisen rajan, päivitetään kaikkien mittapisteiden historiatietojen senhetkinen tilanne, lasketaan kyseisen aikasiivun tunnusluvut ja varmuuskopioidaan tietorakenne massamuistiin.

Joka kerta sovelluksen käynnistyessä tietorakenne valmiiksi laskettuine tunnuslukuineen luetaan varmuuskopiosta takaisin tietorakenteeseen.

Tietorakenteessa pidetään yllä samoja neljää parametria, jotka saadaan Tielaitoksen www-palvelusta: numeerisina arvoina esitetään keskimääräinen nopeus ja liikennetiheys sekä sanallisena kuvauksena liikennemäärä ja liikenteen sujuvuus. Aikasiivun tunnuslukuina käytetään historiatiedoista laskettuja kunkin parametrin mediaaneja.

Asiakkaiden kysellessä mittapisteen historiatietoja, mediaanit voidaan antaa suoraan muistista, mikä helpottaa palvelun skaalautumista. Mediaanien laskentahan on kohtalaisen työläs operaatio, joka edellyttää tietojen järjestämistä.

Aivan kaikkia mittaustuloksia ei kuitenkaan kelpuuteta historiatietoihin: Ensinnäkin mittapisteen mittaustiedot lisätään aikasiivun historiatietoihin vain, jos kyseisen mittapisteen liikennemäärä ylittää puolet kyseisen mittapisteen kyseisen aikasiivun aiemmasta mediaanista. Tällä pyritään suodattamaan vapaapäivät pois ruuhkaisten teiden historiatiedoista. Toiseksi mittaushetken aikaleima ei saa olla yhtä tuntia vanhempi. Tämä lisäys tehtiin sen jälkeen, kun todettiin sovelluksen hakeneen tietoa internetiin liittyvistä välimuisteista. Molemmat parametrit on muutettavissa Windowsin rekisteriä muokkaamalla.

3.5.6 Historiatiedon luku

Historiatietoja haettaessa annetaan aina se kellonaika, jonka mittaustulokset tahdotaan historiatiedoista noutaa. Kyseinen kellonaika osuu harvoin täsmälleen aikasiivun vaihtohetkeen, joten tietoja noudettaessa tehdään interpolointi vierekkäisten aikasiivujen välillä.

3.5.7 Hälytysten antaminen

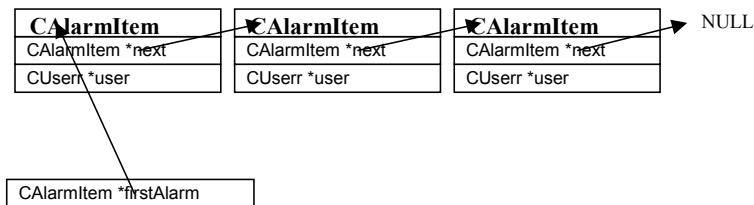
Hälytykset annetaan adaptiivisesti, siis suhteutettuna kunkin mittapisteen viimeaikaiseen liikennetilanteeseen samana kellonaikana.

Kun mittapisteiden tiedot on haettu Tielaitoksen palvelusta, verrataan liikenteen keskimääräistä nopeutta kyseisen ajanhetken historiatietoihin. Jos nopeus on 30% pienempi kuin kyseisen ajanhetken historiatieto ja sujuvuuden sanallinen kuvaus on mitä tahansa muuta kuin ”sujuvaa”, tulkitaan liikenteen tukkeutuneen. Vertailussa ei siis käytetä absoluuttisia arvoja, vaan *oletetaan käyttäjän tuntevan oman työmatkareittinsä*. Hälytys annetaan vain poikkeuksellisen tukkoisesta liikenteestä; joillain reiteillähän liikenne matelee tiettyinä kellonaikoina lähes joka päivä.

Jos liikenteen todetaan jossain mittapisteessä tukkeutuneen, seuraava tehtävä on selvittää, keille asiasta pitää ilmoittaa. Aiemmin kuvassa 11 esitettiin, kuinka käyttäjien määrittelemät triggerit yhdistävät teiden mittaustiedon ja asiakkaiden personointitiedon toisiinsa: Kullakin mittapisteellä on siis linkitetty lista, jonka alkioista lähtee osoitin niihin triggereihin, jotka käsittelevät kyseistä mittapistettä. Kustakin triggeristä on puolestaan osoitin edelleen siihen käyttäjään, joka triggerin on asettanut.

Hälytyksen tapahtuessa käydään läpi kaikki mittapisteen triggerit ja tehdään vertailu, täsmääkö senhetkinen kellonaika tietyllä tarkkuudella (määriteltävissä rekisteritietoihin) triggeriin määritellyn kellonajan kanssa. Jos aika täsmää, voidaan olettaa käyttäjän törmäävän kohta poikkeuksellisen pahaan ruuhkaan, jollei ilmoitusta anneta. Seuraavaksi tarkistetaan, ettei kyseisestä triggeristä ole tällä haavaa jo annettu hälytystä. Tietojahan kerätään viiden minuutin välein, joten pitkään tukkeutunut liikenne aiheuttaisi lukuisan määrän hälytyksiä käyttäjän WAP-puhelimeen.

Jos hälytys pitää antaa käyttäjälle, tehdään se siten, että triggeristä käyttäjään osoittava osoitin (*owner*) kopioidaan kuvassa 13 esitettyyn hälytyslistaan luodun uuden alkion kenttään *user*. Uutta alkioita ei luoda, jos käyttäjä on jo listalla, sillä käyttäjä saa yhdenkin hälytyksen mukana tiedot kaikista personointiinsa liittyvistä mittapisteistä.



Kuva 13. Hälytyslistaan kerätään tietojen päivityksen yhteydessä kaikki ne käyttäjät, joille pitää lähettää hälytys edessä olevasta ruuhkasta.

Kun kaikkein mittapisteiden tiedot on päivitetty, tuhotaan hälytyslista yksi alkio kerrallaan. Ennen alkion tuhoamista massamuistiin kirjoitetaan raportti sen käyttäjän personoiman reitin tiedoista, johon alkiossa viitataan. Raportit kirjoitetaan erilliseen hälytyshakemistoon, josta WAP-palvelun käyttöliittymä voi ne käydä hakemassa.

3.5.8 Käyttäjän esittämiin kyselyihin vastaaminen

Kuten luvussa 3.2 esitettiin, käyttäjä voi sekä pyytää palvelulta personoimansa reitin tilanteen että hakea reittiin tietyllä ajanhetkellä liittyvän historiatiedon. Pyyntö tietokannalle esitetään siten, että käyttöliittymä kirjoittaa tiettyyn hakemistoon (määriteltävissä Windowsin rekisteristä) komentotiedostoja, jota käydään pollaamassa kaksi kertaa sekunnissa.

Komentotiedosto on käyttäjäkohtainen ja nimetty siten, että tiedostonimen alkuosana on käyttäjän tunniste (vrt. kuva 10) ja loppuosana tarkenne .CMD. Komentotiedosto 0500606699.CMD on siis peräisin käyttäjältä 0500606699. Vastaus komentoon kirjoitetaan samaan hakemistoon muuten samannimisenä tiedostona, mutta tarkenne on .RSP. Komentoa 0500606699.CMD seuraa siis vastaus 0500606699.RSP. Esimerkki vastaustiedostosta on kuvassa 14.

```

20;0;45;Tammisto;Helsinkiin;09:28;Sujuvaa;95;1896;Kohtalaisesti
34;0;Kehä 1;Länsi-Pakila;Tapiolaan;09:25;Sujuvaa;87;1908;Kohtalaisesti
36;0;Kehä 1;Kannelmäki;Tapiolaan;09:25;Sujuvaa;85;1452;Kohtalaisesti
38;0;Kehä 1;Konala;Tapiolaan;09:31;Sujuvaa;80;1608;Kohtalaisesti

```

Kuva 14. Vastaustiedoston tiedostoformaatti on sama kuin raporttiedostossakin

Historiatiedot saadaan haettua siten, että komentotiedosto sisältää merkkijonon "history hh:mm", jossa hh:mm kuvaa kellonaikaa. Tällöin käyttäjä saa personoitujen pisteidensä kyseistä kellonaikaa vastaavat historiatiedot. Historiatietojen tulostus tehtiin esimerkinomaiseksi, käytettävyytstutkimuksilla päästäisiin varmasti hyödyllisempiinkin tulostuksiin. Mainittakoon tietorakenteen kätevyydestä, että historiatietojen tulostuksen ja komentotiedoston parseroinnin ohjelmoitiin kului aikaa alle 30 minuuttia.

3.5.9 Skaalautuvuustarkasteluja

Tietojen haku ja parserointi vaatii aina saman laskentamäärän riippumatta käyttäjien määrästä, samoin historiatietojen päivittäminen ja hälytysten analysointi sekä laskennallisesti kenties raskain osuus – mediaanien laskenta.

Kun liikenteen on havaittu jumiutuneen jossain mittapisteessä, asiaankuuluvien käyttäjien tietorakenteet löytyvät suoraan linkitetyn listan osoittimien avulla. Hälytettävien käyttäjien lukumäärä on tilastollisesti suorassa suhteessa palvelun käyttäjien kokonaismäärään. Linkitetyn listan ja osoittimien seuraaminen on kuitenkin laskennallisesti kevyttä. Hälytyslistan laatiminen muistin varaamisineen ja vapauttamisineen kuluu toki aikaa, mutta sen avulla voidaan varmistua, ettei samalla tietojen päivityskerralla lähetetä yhdelle käyttäjälle useita hälytyksiä. Hälytyslistan laatiminen siis minimoi kovalevylle kirjoittamisen määrän, eli prosessin hitaimman toimenpiteen. Enimmäismäärä kovalevylle kirjoitettavia hälytystiedostoja on sama kuin käyttäjien kokonaismäärä.

Koska hälytykset pitää saada kirjoitetuksi ennen seuraavaa tietojen päivitystä, eli viiden minuutin kuluessa, ei hälytysten lähettäminen muodosta pullonkaulaa suurellekaan käyttäjämäärälle. Lisäksi yhdestä mittapisteestä ei tuoteta hälytystä liikenteen jumiutuessa kuin yhden kerran, mikä samalla helpottaa tulevan WAP-push -palvelun kapasiteettitarvetta ja säästää myös käyttäjien hermoja.

Skaalautuvuutta ajatellen työläin vaihe on komentotiedostojen luku ja käsittely. Käyttäjätunnus ilmenee komentotiedoston nimestä, ja vastaava tunnus tulee etsiä myös tietokannasta. Kyseessä on aina merkkijonon vertailu, joten kyselyryöppy tietokantaan, jossa on suuri määrä käyttäjätunnuksia hidastaa palvelua. Käyttäjien toimintaa voidaan toki ohjata palvelun hinnoittelupolitiikalla.

Yhden käyttäjän tiedot vievät tietokannasta 29 tavua, josta 21 tavua on varattu käyttäjän tunnisteelle. Tämän lisäksi kukin triggeri vie 20 tavua. Hälytyslista, jossa pahimmassa tapauksessa ovat mukana kaikki käyttäjät, vie 8 tavua per käyttäjä. Nykyisten Windows NT -palvelimien satojen megatavujen keskusmuisti on käytettävissä, eikä siis ole käyttäjämäärää rajoittava tekijä.

Toteutettu sovellus on optimoitu nimenomaan demossa esitettäville toiminnoille, joten sen voidaan perustellusti olettaa olevan nopeampi kuin mikään samaan tarkoitukseen sovitettu yleiskäyttöinen relaatiotietokanta.

3.6 Palvelun toimivuus ja havaitut seikat

Palvelun demoversio saatiin toimivaksi.

Tielaitoksen sivuilta haetussa tiedossa on noin 20 minuutin viive, mikä aiheuttaa hälytyksen puuttumisen tilanteessa, jossa liikene jumiutuu yhtäkkisesti esimerkiksi onnettomuuden seurauksena. Historiatietojen vertailukohtana kuitenkin on aikaisempina päivinä samaan aikaan käytettävissä olleet liikennetilannetiedot: Mikäli liikenne hidastuu pidemmällä aikavälillä vaikuttavien tekijöiden, kuten joukkoliikennelakon tai

lumipyryn vuoksi, hälytys voidaan päätellä kaksikymmentä minuuttiakin vanhoista tiedoista.

Tieinfoa suoritettaessa havaittiin sovelluksessa kiusallinen virhe, joka sekoitti historiatietoja ainakin osittain: Sovellus luki ajankohtaisena pitämänsä liikennetilanteen joko oman Windowsinsa tai verkon välimuistista. Testausvaiheessa Tieinfon käsittelemää dataa verrattiin internet-selaimen näyttämään. Tällöin välimuistit päivittyivät. Heti, kun testaus lopetettiin ja auki ollut internet-selain suljettiin, mittaustiedot jumiutuivat paikalleen ja päivittyivät vain aika ajoin. Kun tietoja seuraavan kerran vertailtiin, internet-selain avattiin, ja kaikki näytti olevan kunnossa. Tekijällä oli kyllä sellainen hytinä, että *jotain* on pielessä, mutta ei tarkalleen, *mitä*.

4 Johtopäätökset

Demon kaltainen palvelu voidaan toteuttaa hyvinkin yksinkertaisella ohjelmistolla ja pienillä laiteinvestoinneilla.

Teknisenä puutteena demossa on WAP-formaattisen push-palvelun puuttuminen, sillä sitä ei ollut työryhmän käytettävissä.

Alkuperäisenä visiona ollut GSM-tukiasematiedon saaminen palveluun ei välttämättä ole niin yksinkertaista, miltä se kuulostaa. GSM-verkko on tehty etupäässä puheluiden välittämiseen, eikä asiakkaan paikallistamiseen.

GSM:ään suunnitteilla oleva paikannus puolestaan on laskennallisesti sen verran raskas, ettei sitä sellaisenaan voi käyttää autoilijoiden jatkuvaan paikantamiseen. Se on myös tarpeettoman tarkka.

5 Suositukset

5.1 Personointi

Tuotantosovelluksessa asiakas voisi määritellä päivittäisen työmatkareittinsä vaihtoehtoisesti joko WAP:n tai internetin kautta. Asiakas määrittelee jokaiselle reitille myös arvioidun lähtöajan, jonka mukaan mittapisteiden arvioidut ohitushetket saadaan automaattisesti laskettua. Määriteltävänä voisivat olla myös aktiiviset päivät kullekin reitille.

4.25.2 Tekstiviestien ja puhepostilaatikon käyttö

Niin kauan, kun WAP-push ei ole käytettävissä, hälytys kannattaa tarjota SMS-viestinä. Tästä huolimatta tarkemmat tiedot personoidusta reitistä saisi edelleen demon kaltaisella WAP-palvelulla.

Myös senkaltaisen palvelu voisi tulla kyseeseen, jossa hälytys lähetettäisiin SMS-viestinä ja käyttäjän puhepostilaatikossa odottaisi viesti reitin ruuhkista. Viesti poistuisi laatikosta automaattisesti esimerkiksi tunnin kuluessa lähetyshetkestä. Kyseinen malli antaisi muutamia etuja:

- 1) Laskutus tapahtuisi käyttäjän kuunnellessa viestin, eli vain tarpeen mukaan
- 2) Ajantasainen käyttöliittymä olisi verbaalinen. Ajaminen itsessään on visuaalinen toiminta, jonka häiritseminen ylimääräisen näytön katselulla vaarantaa ajosuorituksen. Visuaalisuuden ongelma pätee muuten myös suunniteltuihin autonavigointijärjestelmiin, joissa on graafinen näyttölaite kojelaudan päällä. Verbaalinen informaatio ei häiritse läheskään yhtä paljoa.

4.33) Päätelaitteena voi olla myös vanhempi GSM-puhelin. Reaaliaikaisuuden parantaminen

Palvelun reaaliaikaisuutta voidaan parantaa ennustemallien käytöllä, jolloin ruuhkan kehittyminen voidaan ennakoida, vaikka mittaustiedot saataisiinkin pienellä viiveellä.

4.45.3 Hinnoittelumalleja

Palvelu kyetään hinnoittelemaan esimerkiksi kuukausiperusteisena. Pienten investointi- ja oletettavasti myös pienten käyttökustannusten vuoksi voisi ajatella myös mallia, jossa vaikkapa kolmea mittapistettä tarkkaileva peruspalvelu olisi ilmainen, ja maksua vastaan saisi käyttöönsä ennustemalliin pohjautuvan liikennearvion, enemmän personoitavia mittapisteitä ja kenties myös WAP-pohjaisen personointimahdollisuuden. Yksittäisiä tarpeita varten voidaan käyttää asiakkaan tekemään kyselyyn perustuvaa kertahintaa. Palvelu on sisällöltään sellainen, että se voisi olla kiinnostava hyvinkin laajalle käyttäjäkunnalle.

Formatted: Bullets and Numbering

Formatted: Bullets and Numbering

Formatted: Bullets and Numbering

Palvelun mahdollisessa suosiossa hinnoittelu on tärkeässä asemassa. Mikäli palvelun toteutetaan jokin julkinen yhteisö tai yhtiö, esimerkiksi tielaitos tai YTV, voitaisiin se hinnoitella edulliseksi, jolloin suuretkin käyttäjämäärät ovat mahdollisia.

Palvelu voi myös toimia sponsoripohjaisena sekä paikkatietoon perustuvan kohdennetun markkinoinnin apuvälineenä – ottamalla ilmaispalvelun käyttöönsä asiakas suostuisi paikkatietonsa julkistamiseen kolmannelle osapuolelle, ainakin työmatkareittinsä osalta.

4.55.4 Tukiasematiedon käyttäminen

Huolimatta tarvittavien rajapintojen puuttumisesta, sen tiedon saaminen, minkä tukiaseman piirissä WAP-puhelin sijaitsee, kuulostaa houkuttelevalta. Vaikka tarkkuus ei olekaan henkilökohtaiseen navigointiin riittävä, se riittäisi kymmeniä kilometrejä ajavalle työmatkalaiselle. Helpoiten personointi sujuisi siten, että asiakas laittaisi kotoa lähtiessään päälle seurannan, joka tämän esimerkkimatkan aikana rekisteröisi käytetyt tukiasemat.

Sama ajatus edelleen kehitettynä johtaisi siihen, että varsinaista tiemittausta ei välttämättä enää tarvittaisi, vaan reaaliaikainen tieto saataisiin muiden tavallisesti samaa reittiä ajavien, aikaisemmin liikkeelle lähteneiden liikeistä: Jos seuraavan tukiaseman piiriin ei saavukaan tilastollisesti riittävää määrää reitin käyttäjiä, voitaisiin hälytys antaa perässäajajille.

Formatted: Bullets and Numbering

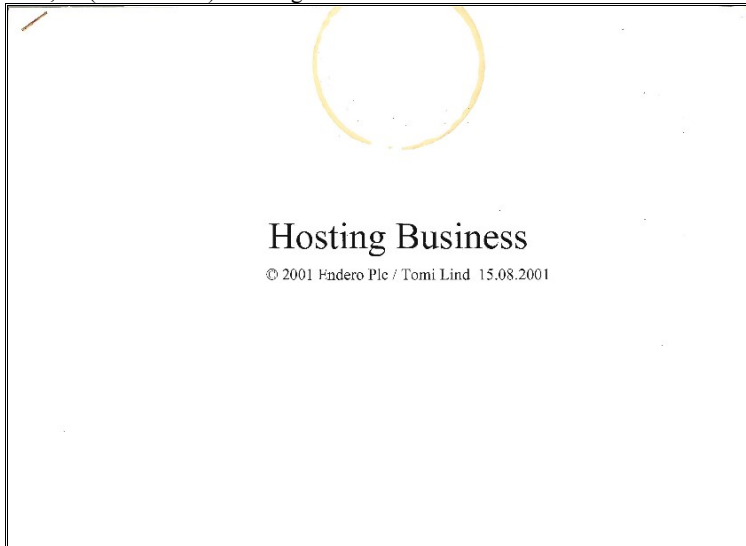
6 Tutkielman arviointia

Tutkielma tehtiin kahden hengen ryhmänä, joka kykeni jakamaan työn kahteen kohtalaisen itsenäiseen kokonaisuuteen. Ryhmään oltaisiin kaivattu lisää henkilöitä, sillä jo nyt tehty työ vei resurssit käytettävyytutkimukselta ja siihen pohjautuvalta liiketoimintamallien kehittämiseltä.

Tehty tutkielma voisikin siten saada jatkoa muiden opintoviikkonälkäisien (jatko-) opiskelijoiden tekemänä.

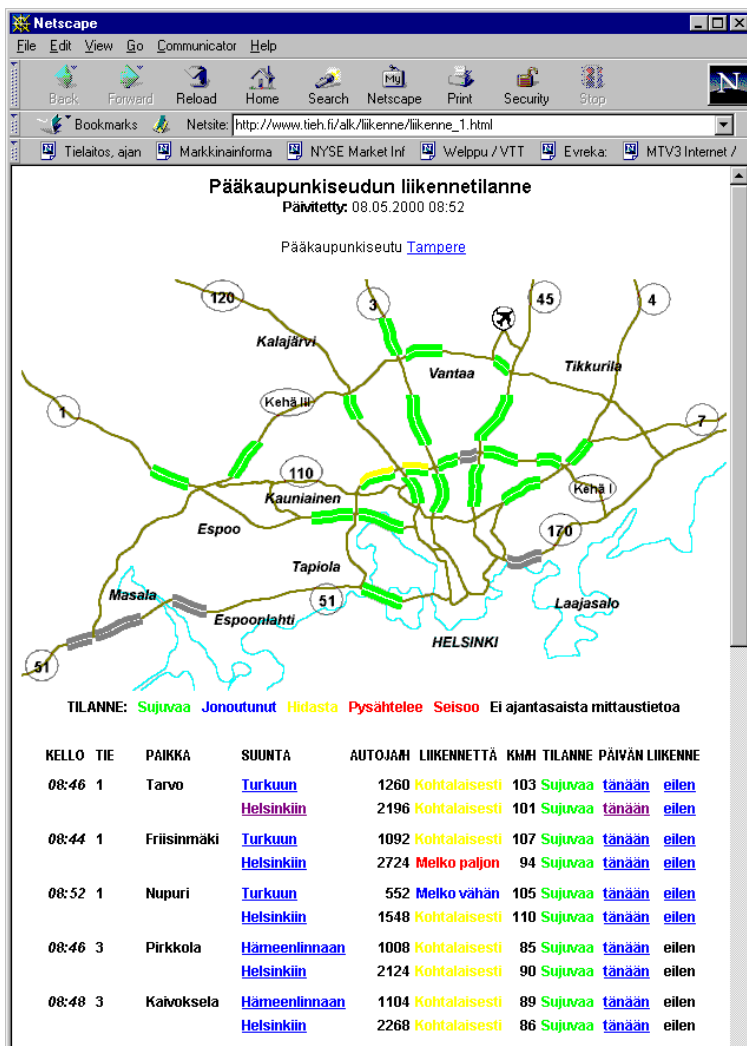
7 Lähdeluettelo

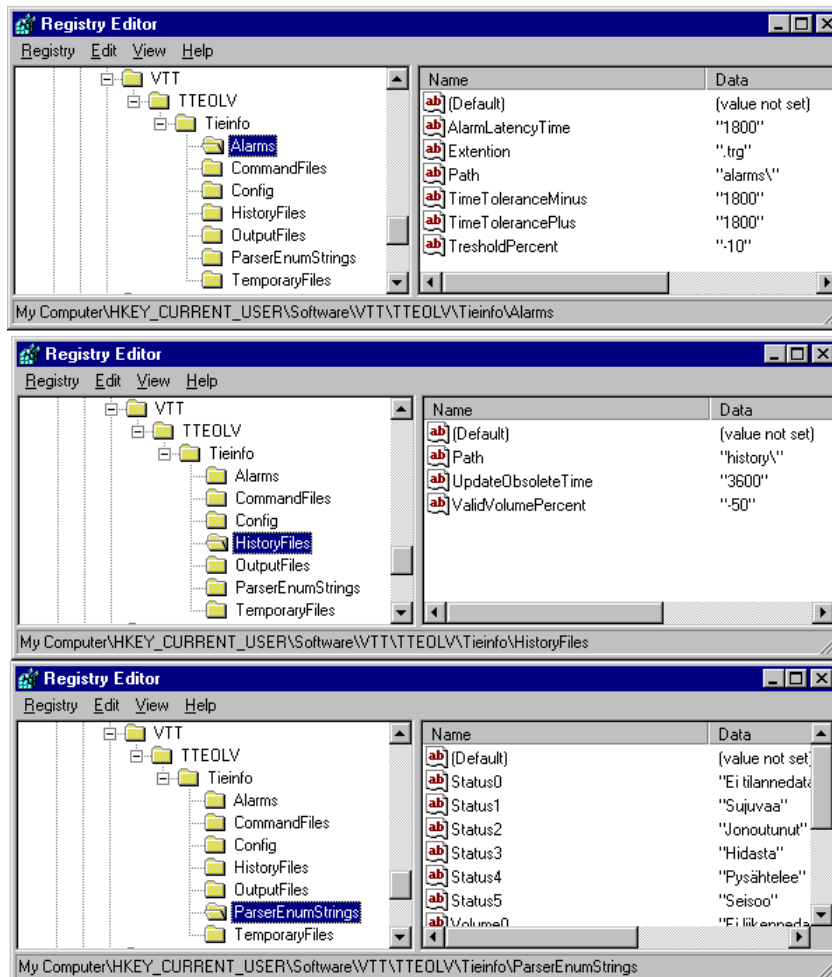
Lind, T. (15. 08 2001). Hosting Business. Helsinki.



8 Liitteet

Liite A, Kuva Tielaitoksen www-palvelusta



Liite B, Otteita Tieinfo rekisteritiedoista

Liite C, Oteita Tieinfon ohjelmakoodista

**** RoadDB.h ****

```
#ifndef __RoadDB_h
#define __RoadDB_h

#include "Tiedefs.h"
#include <time.h>

// Traffic measurement data
struct CMeasurement
{
    public:
        CMeasurement(); // constructor
        long daySec; // timestamp
        int carsPerHour;
        int speed;
        ETrafficVolume trafficVolume; // verbal description of carsPerHour -> e
        ETrafficStatus trafficStatus; // verbal description of traffic fluentnes
        void setTrafficVolume(char *description); // sets enum volume from strin
        void setTrafficStatus(char *description); // sets enum status from strin
        void setTime(char *timeString); // sets timestamp from hh:mm string
        char *getTrafficVolume(); // returns verbal description of enum volume
        char *getTrafficStatus(); // returns verbal description of enum status
        char *getTimeString(); // returns timestamp in hh:mm string format
        char *getSpeedString(); // returns speed as string, or "ei dataa"
        char *getCarsPerHourString(); // returns cars per hour as string, or "e
        CMeasurement &operator=( CMeasurement & );
};

// history data for one course for one time-of-day slice
class CHistoryTimeSlice
{
    public:
        long nominalDaySec; // timestamp
        CMeasurement measurementHistory[HISTORY_DAYS]; // measurement data for 1
        CMeasurement medianValue; // pre-calculated median values
        void addMeasurement(CMeasurement measurement); // add and save
        void calculateMedian(); // calculate median from history data
};

// history data for one course
class CHistoryData
{
    public:
        CHistoryData(void *course); // constructor
        ~CHistoryData(); // destructor
        int saveData(); // save image to disk
        CString historyFileName(void *parentCourse); // returns history file name
        char fileName[3*NAMELEN+20]; // storage for history file name
        long updateSec; // timestamp of last update
        CHistoryTimeSlice timeSlice[(ONE_DAY/HISTORY_INTERVAL)+1]; // data
        CMeasurement getMedian(long daySec); // return interpolated median value
};

// List of all users related to active alarms
class CAlarmItem
{
    public:
        CAlarmItem(); // constructor
        ~CAlarmItem(); // destructor
};
```


**** RoadDB.cpp ****

```
#include "stdafx.h"

#include <stdio.h>
#include <io.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>
#include "RoadDB.h"
#include "Tiedefs.h"
#include "UserDB.h"

extern STieConfig Config; // config data from the registry

// ***** CHistoryData functions *****

// CHistoryData constructor:
// CHistoryData holds the history information of its parent course.
// Constructor generates file name from the parent course data and retrieves
// the data from the file. If not successful, it informs the user that the
// history data is to be initialized. The file name is stored to the structure
// for subsequent file save operations.

CHistoryData::CHistoryData(void *parentCourse)
{
    int retvalue=1; // default no success
    strcpy(fileName,historyFileName(parentCourse)); // generate file name
    CFile historyFile;
    if(historyFile.Open(fileName, CFile::modeRead|CFile::typeBinary|CFile::s
    {
        if(historyFile.Read(this,sizeof(CHistoryData))!=sizeof(CHistoryData))
        {
            CString message="Error in reading history file.";
            message+=historyFileName(parentCourse);
            AfxMessageBox(message);
        }
        else
        {
            retvalue=0; // success
        }
        historyFile.Close();
    }
    if(retvalue)
    {
        CString message;
        message="Initializing historyfile ";
        message+=historyFileName(parentCourse);
        AfxMessageBox(message);
    }
};

/* update version
// This is a skeleton for the constructor in the case the stucture of CHistory
// changed.
CHistoryData::CHistoryData(void *parentCourse)
{
    int retvalue=1;
    unsigned int oldSize=sizeof(CHistoryTimeSlice)-sizeof(CMeasurement);
    strcpy(fileName,historyFileName(parentCourse));
```